

DATABRIDGE ACADEMY

ZERO TO DATA ANALYST

20-WEEK
HANDS-ON
COURSE

SQL, Excel, Power BI and Python,
step by step, for complete beginners

SQL

EXCEL

POWER BI

PYTHON



Real retail dataset · Exercises & answers · Ages 15+

Zero to Data Analyst (Free Sample)

SQL, Excel, Power BI and Python, Step by Step, for Complete Beginners

DataBridge Academy

First Edition

Zero to Data Analyst (Free Sample): SQL, Excel, Power BI and Python, Step by Step, for Complete Beginners

First Edition, 2026

Copyright © 2026 DataBridge Academy. All rights reserved.

No part of this book may be reproduced, stored or transmitted in any form or by any means without written permission from the publisher, except for brief quotations in reviews. The practice files may be used freely by the purchaser for personal learning and teaching.

Disclaimer. This book is provided for educational purposes. Every effort has been made to ensure accuracy, but software changes over time and menus may differ in your version. The author and publisher accept no liability for any loss arising from the use of this book.

Fictional data. BrightMart and all of its stores, customers, employees and figures are fictional and were generated for teaching. Any resemblance to real businesses or people is coincidental.

Trademarks. MySQL is a registered trademark of Oracle and/or its affiliates. Microsoft, Excel, Power BI, Power Query, Windows and VS Code are trademarks of the Microsoft group of companies. Tableau is a trademark of Salesforce, Inc. Python is a trademark of the Python Software Foundation. Jupyter is a trademark of LF Charities. GitHub is a trademark of GitHub, Inc. These names are used for identification only; this book is independent and is not affiliated with, sponsored by or endorsed by these companies.

Practice files: github.com/ugochiya21/zero-to-data-analyst

Published by DataBridge Academy.

Contents

How to use this book	5
PART A: SQL with MySQL	10
Chapter 1: Data, databases, and your first SQL commands	11
Chapter 2: Filtering data with WHERE	24
Enjoyed the sample?	31

How to use this book

Read this first: it takes five minutes

This book takes you from **knowing nothing about data** to working like a **junior data analyst**. It's written for complete beginners: every command is explained word by word, with what it does and when you'd use it, and you practise on a realistic company dataset from the very first chapter.

You'll learn the five things employers ask for, in this order:

Part	Tool	What it's for	Weeks
A	SQL (MySQL)	Getting data out of company databases	1-4
B	Excel	Quick analysis, PivotTables, dashboards	5-7
C	Power BI	Professional interactive reports for managers	8-10
D	Python	Automation, cleaning, deeper analysis, charts	11-15
E	Statistics, Python power tools, intro to machine learning, Tableau, GitHub, career	Thinking like an analyst, predicting, and getting hired	16-20

Who this book is for

- **Complete beginners** who want a career in data: no maths degree or coding experience needed.
- **Graduates and career changers** who want a practical portfolio, not just theory.
- **Professionals** in finance, sales, operations, HR or banking who want to stop copying and pasting in Excel and start analysing properly.
- **Secondary-school students (age 15+)** who are curious about tech careers. See the student track below.
- **Teachers and bootcamp instructors** looking for a ready-made 20-week programme with exercises and answers.

Start here if you're in secondary school

Data skills look excellent on university applications and CVs, and you can start now. You don't need to do the whole book. Follow the **Student Track** at about **1 hour a day**:

Step	Chapters	What you'll be able to do
1	1-3	Ask a database questions and summarise thousands of rows
2	4-5	Combine tables and create your own categories
3	10, 11, 13	Use Excel like a professional: formulas, PivotTables, charts
4	20, 21, 24	Write your first Python and make charts
5	27 (sections 27.1 and 27.6)	Understand averages, spread and correlation

Step	Chapters	What you'll be able to do
6	Mini-project	Answer 5 questions about BrightMart and present them as 5 slides to your family or class

Tips for students:

- Ask a parent or guardian to help install the free software (Chapter 1, Section 2).
- If something feels hard, skip ahead to the exercises, try them, then read the section again. That's how professionals learn too.
- Try the same ideas on data you care about: your football team's results, your class test scores (anonymised), or your own phone screen-time.

For parents and teachers: the book uses a fictional shop's sales data. There's no unsuitable content, and every tool used is free (except Excel, which most schools already have).

What you need

Item	Notes
A Windows PC or laptop (8 GB RAM recommended)	Mac users can do the SQL, Python and Excel parts; Power BI Desktop runs only on Windows
MySQL Server + MySQL Workbench	Free. Installation is in Chapter 1
Microsoft Excel 2021, 2024 or Microsoft 365	Needed for XLOOKUP and dynamic arrays (Chapter 11). Older Excel works for most of Part B
Power BI Desktop	Free, from the Microsoft Store (Chapter 15)
Python 3 + Jupyter	Free. Installation is in Chapter 20
Internet	Only for downloading the software and practice files

Get your practice files

Everything you practise on is in one free download:

github.com/ugochiya21/zero-to-data-analyst

Unzip it into a folder you'll remember, for example **Documents → DataAnalysis**. This book calls it your **practice-files folder**. It contains:

File	What it is
<code>brightmart.sql</code>	The full BrightMart database. Load it into MySQL in Chapter 1
<code>marketing_campaigns.sql / .csv</code>	An extra table for practising imports
<code>data*.csv</code>	Every table as a CSV file, for Excel, Power BI and Python
<code>data\sales_flat.csv</code>	Ready-made sales lines (one row per item sold)
<code>data\supplier_orders_messy.csv</code>	A deliberately messy file for cleaning practice
<code>data\bm_online_orders.json</code> , <code>data\exchange_rates_backup.json</code>	JSON files for Chapter 28 (web data and APIs)

The BrightMart story

Throughout the book, you are a **junior data analyst at BrightMart**, a fictional Nigerian retail chain. It has 7 shops (Lagos x2, Abuja, Port Harcourt, Ibadan, Kano, Enugu) and an online store. It sells electronics, groceries, home and kitchen items, fashion, health and beauty products, and baby products.

Every chapter uses **the same data** in a different tool. By the end, you'll have answered the same business questions in SQL, Excel, Power BI and Python. That's exactly the kind of portfolio employers want to see.

BrightMart, its stores, customers, staff and figures are entirely fictional, generated for teaching. Any resemblance to real businesses or people is coincidental.

The data at a glance

Table	Rows	One row =
stores	8	a branch
employees	37	a staff member
customers	1,500	a registered customer
products	81	an item for sale
orders	13,232	a receipt (Jan 2023 – Dec 2025)
order_items	23,747	one product line on a receipt
returns	659	a returned item
marketing_campaigns	24	a marketing campaign

The most important formula in this book. The money from one order line is:

```
line revenue = quantity × unit_price × (1 – discount)
```

discount is stored as a fraction: 0.10 means 10% off. Only orders with **status = 'Completed'** count as real sales. Cancelled and Pending orders are not revenue.

Your daily routine (3 hours)

Time	Activity
0:00–0:45	Learn: read the chapter section and type every example
0:45–2:15	Practise: do the exercises without looking at the answers
2:15–2:45	Business challenge: answer the "manager's request"
2:45–3:00	Recap: write 5 things you learned in a notebook, in your own words

Less time available? Do the same steps at 1 hour a day; the course just takes longer.

Rules that make you good fast:

1. **Type everything.** Never copy and paste examples.
2. **Break things on purpose.** Remove a comma and read the error. Errors are teachers.

3. **Answer in English, not just numbers.** "Online made \$318m, 24% of all revenue" beats "318343335".
4. **Check your answers only after trying.** Answers are at the end of each chapter.
5. **Save your work** (Ctrl + S) and keep a portfolio folder of every project.

Course map: which chapter on which day

Week	Days	Chapter
1	1-2	1. Data, databases and first SQL commands
1	3-4	2. Filtering with WHERE
1-2	5-7	3. Summarising: COUNT, SUM, GROUP BY, HAVING
2	8-10	4. Joining tables
3	11-12	5. Functions: text, numbers, dates and CASE
3	13-14	6. Subqueries, CTEs and UNION
3-4	15-17	7. Window functions
4	18-19	8. Cleaning data, changing data, views
4	20-24	9. SQL project: BrightMart business review
5	25-27	10. Excel foundations and getting data in
5-6	28-31	11. Excel formulas every analyst uses
6	32-33	12. Cleaning data with Power Query
6-7	34-36	13. PivotTables and charts
7	37-42	14. Excel project: sales dashboard
8	43-45	15. Power BI: getting started and Power Query
8-9	46-48	16. Data modelling: star schema and relationships
9	49-52	17. DAX measures
9-10	53-55	18. Visuals and report design
10	56-60	19. Power BI project: executive report
11	61-64	20. Python foundations
12	65-68	21. pandas: loading, selecting, filtering
12-13	69-72	22. pandas: grouping, merging, dates
13	73-75	23. Cleaning data in pandas
14	76-79	24. Charts with matplotlib and seaborn
14	80-82	25. Python + MySQL + Excel automation
15	83-90	26. Python project: customer analysis
16-17	91-98	27. Statistics for analysts
17	99-102	28. Python power tools: NumPy, errors, JSON, APIs, Plotly
18	103-107	29. A first taste of machine learning
18-19	108-110	30. Tableau, Git and GitHub
19-20	111-120	31. Capstone project and getting hired

If a chapter takes you longer, that's fine. **Understanding beats speed.**

Saving your SQL work (read before Chapter 1)

Your **data** is saved automatically inside MySQL, but the **queries you type** are not. Save them as files:

1. Make a folder `my_sql_work` inside your practice-files folder.
2. In Workbench, press **Ctrl + S** and save each chapter's work as `ch01_exercises.sql`, `ch02_exercises.sql`, ...
3. Before you stop, add a bookmark comment: `-- STOPPED HERE: Chapter 2, section 2.4.`
4. Next time: **File → Open Recent**.

Lost something you didn't save? In Workbench, change the **Output** panel's dropdown from **Action Output** to **History Output**. It lists every query you ran, by date.

PART A

SQL with MySQL

Getting data out of databases

Chapter 1: Data, databases, and your first SQL commands

Course days 1-2 · Complete beginner. Nothing is assumed.

Part	Time	What you will learn
1	0:00-0:25	What data analysis is and what a database is
2	0:25-0:55	Installing and opening your tools
3	0:55-1:25	Importing data: a <code>.sql</code> file and a <code>.csv</code> file
4	1:25-2:20	Your first commands: <code>SELECT</code> , <code>FROM</code> , <code>*</code> , <code>AS</code> , <code>LIMIT</code> , <code>DISTINCT</code> , <code>ORDER BY</code>
5	2:20-2:40	Exporting data: saving query results to a file for Excel
6	2:40-3:00	Exercises and a quiz you send to me

Read slowly. Type everything yourself; don't copy and paste. Typing is how your fingers and brain learn.

Section 1: The big picture (25 min)

1.1 What does a data analyst do?

A business keeps records of everything: every sale, every customer, every product. A **data analyst** turns those records into **answers** that help managers make decisions.

Manager: "Which store made the most money last December?"

Analyst: pulls the data, calculates the answer, and replies: "Lekki: ₦18.2 million, 22% more than Ikeja."

Every tool you'll learn does the same job in a different way:

Tool	What it's best at
SQL / MySQL	Getting data out of the company database (where the data lives)
Excel	Quick calculations, tables and simple charts
Power BI	Professional, interactive dashboards for managers
Python	Automation, large data, advanced analysis

We start with SQL because **all the data lives in databases**. If you can't get the data out, you can't analyse it.

1.2 What is data?

Data is recorded facts. For example: "Amaka bought 2 bags of rice for ₦12,000 on 5 Jan 2025."

1.3 What is a table?

A table is data arranged in rows and columns, like an Excel sheet.

customer_id	first_name	last_name	city
1	Amaka	Okafor	Lagos
2	Tunde	Bello	Abuja

- A **column** (also called a **field**) is one type of information, such as **city**.
- A **row** (also called a **record**) is one complete item, such as one customer.
- A **primary key** is a column whose value is **unique** for every row, such as **customer_id**. No two customers share the same ID. It's like a matric number or an NIN.

1.4 What is a database?

A **database** is a collection of related tables. Our database is called **brightmart** and has 7 tables:

Table	What ONE row means
stores	one BrightMart branch
employees	one staff member
customers	one registered customer
products	one item we sell
orders	one checkout (one receipt)
order_items	one product line on a receipt
returns	one item a customer brought back

Why split data into many tables? So nothing is typed twice. A customer's name is stored once in **customers**. Every order just stores the **customer_id** number, and we look up the name when we need it. A column that points to another table like this is called a **foreign key**.

```
customers (customer_id) ← orders (customer_id)
stores    (store_id)    ← orders (store_id)
orders    (order_id)    ← order_items (order_id)
products  (product_id) ← order_items (product_id)
```

1.5 MySQL, SQL and Workbench: three different things

- **SQL** (say "sequel" or "S-Q-L", short for *Structured Query Language*) is the **language** you use to talk to a database.
- **MySQL** is the **database software** that stores the data and runs on your PC in the background. You'll install it in Section 2.
- **MySQL Workbench** is the **program you type in**. It sends your SQL to MySQL and shows you the results.

Think of it this way: MySQL is the kitchen, SQL is the language you order food in, and Workbench is the waiter.

A command you send is called a **query**: you are *querying* (asking) the database a question.

Section 2: Setting up your tools (30 min)

Step 1: Install MySQL Server (the database)

1. In your browser, go to dev.mysql.com/downloads/mysql (MySQL Community Server, free).
2. Choose **Microsoft Windows** and download the **MSI Installer** (64-bit). On the next page, click "**No thanks, just start my download.**"
3. Run the installer and choose **Typical**. When the **MySQL Configurator** opens:
 - Keep **Port 3306**.
 - Set a **root password**. **Write it down somewhere safe**. You'll need it every time.
 - Keep "**Configure MySQL Server as a Windows Service**" and "**Start the MySQL Server at System Startup**" ticked.
4. Click **Next / Execute** until it finishes.

Mac users: download the **macOS DMG** from the same page, install it, and set the root password when asked. Everything in the SQL and Python parts of this book works the same on a Mac.

Step 2: Install MySQL Workbench (where you type)

1. In your browser, go to dev.mysql.com/downloads/workbench.
2. Under "Select Operating System", choose **Microsoft Windows** and click **Download** next to the **.msi** file.
3. On the next page, click "**No thanks, just start my download.**"
4. Open the downloaded file and keep clicking **Next**, then **Install**, then **Finish**.

Step 3: Connect to MySQL

1. Open **MySQL Workbench** from the Start menu.
2. Under "MySQL Connections" you should see a grey box called **Local instance** (e.g. *Local instance MySQL84* or *Local instance 3306*). Click it.
 - If there's no box, click the ⊕ next to "MySQL Connections". Set **Connection Name** to **My PC**, **Hostname** to **localhost**, **Port** to **3306** and **Username** to **root**, then click **OK** and click the new box.
3. Enter the **root password** you created when MySQL was installed. Tick "Save password" so you don't have to type it every time.
 - Forgotten it? The easiest fix for a fresh install is to uninstall MySQL Server, delete the folder **C:\ProgramData\MySQL**, and install again with a new password. (You haven't stored any data yet, so nothing is lost.)

Words to know: **localhost** means "this same computer". **root** is the main administrator account in MySQL. **3306** is the "door number" (port) MySQL listens on.

Step 4: Learn the Workbench screen

NAVIGATOR (Schemas tab: your databases)	QUERY EDITOR (you type SQL here)
	[Run] = run everything
	RESULT GRID (your answer appears here)
	OUTPUT (success or error messages)

- **Navigator, Schemas tab (left):** lists your databases. "Schema" is MySQL's other word for database.
- **Query Editor (middle):** where you type.
- **Result Grid:** the table of answers.
- **Output (bottom):** green ✓ means it worked, red ✖ means there's an error and a message explaining it.

Your version may look different. Some newer versions of Workbench have a blue **Run** button at the top instead of the ⚡ lightning icon. They do the same thing. Wherever this book says ⚡, click **Run**.

Keyboard shortcuts you'll use 100 times a day:

- **Ctrl + Enter** runs the statement your cursor is on.
- **Ctrl + Shift + Enter** runs everything in the editor.
- **Ctrl + T** opens a new query tab.

Section 3: IMPORTING data (30 min)

"Importing" means bringing data from a file **into** the database. You'll meet two common file types at work.

3A. Importing a **.sql** file (a database backup / script)

A **.sql** file is a text file full of SQL commands. Running it rebuilds a whole database. **brightmart.sql** does exactly that.

1. In Workbench, click **File → Open SQL Script...**
2. Find and open **brightmart.sql**.
3. The file opens in the editor. Scroll through it and you'll see commands such as **CREATE TABLE** and **INSERT INTO**. Don't worry about understanding them yet.
4. Click the ⚡ **lightning bolt** (or press **Ctrl + Shift + Enter**). Wait 10–60 seconds until the Output panel shows green ticks.
5. In the Navigator, click the ↻ **refresh** icon next to "SCHEMAS". You'll see **brightmart**. Click the ► arrow to expand it, then **Tables**, to see the 7 tables.

What the commands inside the file do:

Command	Meaning in plain English
<code>DROP DATABASE IF EXISTS brightmart;</code>	"If a database called brightmart already exists, delete it" (so we start fresh)
<code>CREATE DATABASE brightmart;</code>	"Make a new empty database called brightmart"
<code>USE brightmart;</code>	"From now on, work inside brightmart"
<code>CREATE TABLE stores (...)</code>	"Make an empty table with these columns"
<code>INSERT INTO stores (...) VALUES (...)</code>	"Put these rows into the table"

Every SQL statement ends with a semicolon ;. It's like a full stop: it tells MySQL that the sentence is finished.

Reading the Output panel while it loads. You'll see lines like:

```
#81: OK, 500 rows affected in 25.141ms
```

This means statement number 81 ran **OK** and put **500 rows** into a table in 25 milliseconds. The file inserts data in blocks of 500 rows, so most lines say 500. Every line should say **OK**. A red **x** means an error.

If a white dot ● appears on the `brightmart.sql` tab, the editor thinks the file has changed. When you close it and Workbench asks to save, click **Don't Save** to keep the original file clean.

Check that the import worked

A professional always checks that all the data arrived. Open a new tab and run:

```
USE brightmart;
SELECT COUNT(*) FROM orders;
```

These are two separate instructions.

Line 1: `USE brightmart;`

Word	Meaning
<code>USE</code>	"Go into" or "work inside"
<code>brightmart</code>	the name of our database
<code>;</code>	end of the instruction, like a full stop

In plain English: "MySQL, from now on I'm working in the brightmart database." Your MySQL can hold many databases (for example `brightmart`, `sakila`, `sys`), and they may contain tables with the same names. `USE` tells MySQL which one you mean, like opening the right folder before looking for a file. The selected database turns **bold** in the Navigator.

Line 2: `SELECT COUNT(*) FROM orders;`

Word	Meaning
<code>SELECT</code>	"Show me"
<code>COUNT(...)</code>	"Count how many..."

Word	Meaning
*	"...rows" (every row, whatever is in it)
FROM orders	"...in the orders table"
;	end of the instruction

In plain English: *"Show me how many rows are in the orders table."* The answer should be **13232**. One row in **orders** is one receipt, so BrightMart had 13,232 orders from 2023 to 2025. We knew the file contained 13,232 orders, so if MySQL shows 13232, nothing went missing.

Now count the other tables by changing only the table name:

```
SELECT COUNT(*) FROM customers;    -- expect 1500
SELECT COUNT(*) FROM products;    -- expect 81
SELECT COUNT(*) FROM stores;      -- expect 8
```

To run one line only: click anywhere on it and press **Ctrl + Enter**. The  / **Run** button runs everything in the tab.

3B. Importing a .CSV file

A **CSV** (*Comma-Separated Values*) file is the most common data file in the world. Open **marketing_campaigns.csv** in Notepad and you'll see this:

```
campaign_id,campaign_name,channel,start_date,end_date,budget_ngn,new_signups
1,New Year Deals 2023,Flyer,2023-01-01,2023-01-21,750000,746
```

The first line is the column names. Each line after that is one row, with values separated by commas. Excel can open it too.

Import it into MySQL:

1. In the Navigator, expand **brightmart**, then **right-click "Tables" → Table Data Import Wizard**.
2. **File path:** Browse and choose **marketing_campaigns.csv**, then click **Next**.
3. Choose **Create new table** and name it **brightmart.marketing_campaigns**. Click **Next**.
4. Workbench shows each column and guesses its **data type** (see below). Check that it guessed these:
 - **campaign_id, budget_ngn, new_signups** → **int**
 - **campaign_name, channel** → **text**
 - **start_date, end_date** → **text** is fine for now. We'll convert them on a later day.
5. Click **Next** → **Next** → **Finish**. It should report **24 records imported**.
6. Refresh the Navigator. There are now 8 tables.

Data types: every column holds only one kind of value.

INT is a whole number (5, 13232). **DECIMAL (12, 2)** is money with 2 decimal places (12500.50). **VARCHAR(40)** / **TEXT** is words (up to 40 characters for VARCHAR(40)). **DATE** is a date written YEAR-MONTH-DAY (2025-12-24).

Check it worked. Type this in the editor and press **Ctrl + Enter**:

```
SELECT * FROM brightmart.marketing_campaigns;
```

You should see 24 rows.

If right-clicking shows no Import Wizard

1. Make sure the **Schemas** tab is selected in the Navigator (not Administration), and click ↺.
2. Make sure **brightmart** is in the list. If it isn't, do Section 3A first: the database must exist before you can import into it.
3. Try right-clicking the word **brightmart** itself, or expand it with ▶ and then right-click **Tables**.
4. Some newer versions of Workbench don't have the wizard at all.

Backup method: import with a SQL script. Open `marketing_campaigns.sql` (File → Open SQL Script → your practice-files folder) and click ↵ / **Run**. The script does by hand exactly what the wizard does behind the scenes:

Step	Command	Meaning
1	<code>USE brightmart;</code>	Work inside our database
2	<code>DROP TABLE IF EXISTS marketing_campaigns;</code>	Delete any half-made copy from earlier attempts, so we start clean
3	<code>CREATE TABLE marketing_campaigns (...)</code>	Make an empty table and give every column a name and a data type
4	<code>INSERT INTO ... VALUES (...), (...);</code>	Put the 24 rows of data into the table. Each (...) is one line of the CSV
5	<code>SELECT * FROM marketing_campaigns;</code>	Check that the data arrived

Once you've refreshed SCHEMAS, brightmart will have **8 tables**.

Section 4: Your first SQL commands (55 min)

Open a new tab (**Ctrl + T**). Type and run each example one at a time with **Ctrl + Enter**, with your cursor on that statement.

Command 1: USE

```
USE brightmart;
```

What it does: chooses which database you're working in. Without it, MySQL doesn't know where your tables are. **When to use it:** once at the top of each new tab. (In the Navigator, the database name turns **bold** when it's selected.)

Command 2: SHOW TABLES and DESCRIBE

```
SHOW TABLES;
DESCRIBE products;
```

What they do: `SHOW TABLES` lists every table in the database. `DESCRIBE products` lists the columns of the products table and their data types. **When to use them:** at the start of any job with a new database, to explore what's there. Professionals always explore

first.

Command 3: **SELECT ... FROM ...** (the most important command in SQL)

```
SELECT * FROM stores;
```

Word by word:

- **SELECT** means "show me".
- ***** (star) means "all columns".
- **FROM stores** means "from the table called stores".
- **;** ends the statement.

In plain English: "Show me every column from the stores table."

Command 4: choosing specific columns

```
SELECT store_name, city FROM stores;
```

What it does: shows only the columns you list, **separated by commas**, in the order you list them. **Why it matters:** real tables can have 100+ columns. Asking only for what you need keeps results clear and queries fast. In professional work, **SELECT *** is mainly for exploring.

Command 5: writing a query neatly

SQL doesn't care about line breaks or capital letters, but people reading your work do. Professionals write it like this:

```
SELECT product_name,  
       category,  
       unit_price  
FROM products;
```

The convention is **COMMANDS IN CAPITALS** and **column_names** in lowercase.

Command 6: **LIMIT**, for looking at only a few rows

```
SELECT * FROM orders LIMIT 10;
```

What it does: returns only the first 10 rows. **When to use it:** **orders** has 13,232 rows, so use **LIMIT** to take a quick peek without loading everything.

Command 7: calculations and **AS** (renaming a column)

```
SELECT product_name,  
       unit_price,  
       unit_cost,  
       unit_price - unit_cost AS profit_per_unit  
FROM products;
```

What it does: **unit_price - unit_cost** calculates a new column for every row. It isn't saved in the table; it only appears in your result. **AS profit_per_unit** gives that new column a name, called an **alias**. Without it, the column header would read **unit_price - unit_cost**.

Maths you can use: **+** add, **-** subtract, ***** multiply, **/** divide.

```
SELECT product_name,
```

```
    unit_price,  
    unit_price * 1.075 AS price_with_vat  
FROM products;
```

(Nigeria's VAT is 7.5%, so price × 1.075 is the price including VAT.)

Command 8: ROUND

```
SELECT product_name,  
       ROUND((unit_price - unit_cost) / unit_price * 100, 1) AS margin_pct  
FROM products;
```

What it does: `ROUND(number, 1)` rounds to 1 decimal place. This calculates each product's **profit margin %**, the share of the selling price that is profit. Analysts use margin all the time.

Command 9: DISTINCT, for removing repeats

```
SELECT category FROM products;           -- 81 rows, many repeated  
SELECT DISTINCT category FROM products;  -- 6 rows, each category once
```

What it does: shows each different value only once. **When to use it:** to answer "what are all the possible values?", such as all the categories or all the payment methods.

The text after `--` is a **comment**. MySQL ignores it. Use comments to leave notes for yourself.

Command 10: ORDER BY, for sorting

```
SELECT product_name, unit_price  
FROM products  
ORDER BY unit_price DESC;
```

What it does: sorts the results.

- **ASC** sorts smallest to largest, or A to Z. This is the default if you write nothing.
- **DESC** sorts largest to smallest, or Z to A.

Sorting by two columns:

```
SELECT category, product_name, unit_price  
FROM products  
ORDER BY category ASC, unit_price DESC;
```

This sorts by category first, then by price from highest to lowest within each category.

Putting it together: a "Top N" question

"What are our 5 most expensive products?"

```
SELECT product_name, unit_price  
FROM products  
ORDER BY unit_price DESC  
LIMIT 5;
```

The fixed order you MUST write clauses in

```
SELECT    columns
```

```
FROM      table
ORDER BY  column
LIMIT     number;
```

(From Chapter 2, **WHERE** goes between FROM and ORDER BY.) If the order is wrong, you'll get an error.

Reading error messages (don't panic!)

Run this deliberately wrong query:

```
-- deliberate mistake: this query is SUPPOSED to fail (ERROR EXPECTED)
SELECT product_nam FROM products;
```

The Output panel shows: **Error Code: 1054. Unknown column 'product_nam' in 'field list'**, meaning there's a spelling mistake. Errors are normal. **Read the message**, because it usually tells you exactly what's wrong.

Common beginner errors:

Error	Usual cause
1054 Unknown column	Column name misspelled
1146 Table doesn't exist	Table name misspelled, or you forgot USE brightmart;
1064 You have an error in your SQL syntax	Missing comma, extra comma, or clauses in the wrong order
1046 No database selected	You forgot USE brightmart;

Clearing the Output panel. After loading a big file, the Output panel fills with hundreds of lines. Right-click inside it and choose **Clear** to empty it. This only removes the messages, not your data. To hide the panel, use the **Toggle output area** button at the top-right of Workbench, or **View → Panels → Hide Output Area**.

Section 5: EXPORTING data (20 min)

"Exporting" means taking a result **out of** the database into a file you can send to your manager or open in Excel.

Method 1: Export a query result to CSV (you'll use this most)

1. Run:

```
SELECT product_name, category, unit_price,
       ROUND((unit_price - unit_cost) / unit_price * 100, 1) AS margin_pct
FROM products
ORDER BY margin_pct DESC;
```

- Above the Result Grid, click the **Export** icon (a small disk with an arrow, labelled "Export recordset to an external file").
- Choose **Format: CSV** and save it as **product_margins.csv** in your Documents folder.
- Open the file in **Excel**. Your query result is now a spreadsheet.

Method 2: Export a whole table

In the Navigator, **right-click a table (e.g. stores) → Table Data Export Wizard**, then **Next**. Choose a file name and the CSV format, then **Next → Finish**.

Method 3: Back up the whole database

Go to **Server → Data Export**, tick **brightmart**, choose **Export to Self-Contained File**, and click **Start Export**. This creates a **.sql** file just like the one you imported. That's how databases are backed up and shared.

Import vs export: import means file → database. Export means database → file.

Section 6: Exercises (20 min, plus homework time if needed)

Start a new tab and write **USE brightmart;** at the top. Write every answer below it. When you're done, save it with **File → Save Script As...** as **day01_answers.sql**.

1. Show all columns of the **employees** table.
2. Show only **first_name**, **last_name** and **role** from **employees**.
3. Show the first 15 rows of **customers**.
4. Use **DESCRIBE** to see the columns of **order_items**. What data type is **discount**?
5. List all the **different** cities in **stores**.
6. List all the different **payment_method** values in **orders**.
7. List all the different **status** values in **orders**.
8. Show each product's name, price, and price **including 7.5% VAT** (call it **price_with_vat**), rounded to 0 decimal places.
9. Show each employee's name, monthly salary, and **yearly salary** (call it **annual_salary**).
10. Show the 10 cheapest products (name and price).
11. Show employees sorted by salary, highest first. Where salaries are equal, sort by **last_name** A to Z.
12. Show the 10 **most recent** orders (hint: sort by **order_date**).
13. From **marketing_campaigns**, show **campaign_name**, **budget_ngn**, **new_signups**, and **cost per signup** (**budget_ngn / new_signups**, rounded to 0 decimals, call it **cost_per_signup**), with the most expensive per signup first.
14. **Export** your result from Q13 to a CSV file and open it in Excel.

Data detective: run **SELECT DISTINCT city FROM customers;** and **SELECT DISTINCT gender FROM customers;**. What problems do you notice? (Hint: look for the same thing written in different ways.)

Quiz: answer in your own words

1. What's the difference between a database, a table, a row and a column?
2. What's the difference between SQL, MySQL and MySQL Workbench?
3. What does ***** mean in **SELECT ***?
4. What does **AS** do?
5. What's the difference between **ASC** and **DESC**?

6. What's the difference between importing and exporting?
 7. Why does every statement end with `;`?
-

✓ When you finish

1. Save your work as `ch01_exercises.sql` (**Ctrl + S**).
 2. Check your answers against the answer key below. Your SQL may look different and still be right; compare the **results**.
 3. For anything you got wrong, re-type the correct query and run it until you understand **why** it works.
 4. Then move on to **Chapter 2: filtering data with WHERE**.
-

Answers: Chapter 1

Try every exercise yourself **before** you look here. Your answer may be written differently and still be correct. What matters is whether you get the same result.

```
USE brightmart;
-- 1
SELECT * FROM employees;
-- 2
SELECT first_name, last_name, role FROM employees;
-- 3
SELECT * FROM customers LIMIT 15;
-- 4 (discount is DECIMAL(4,2): a number with 2 decimal places, e.g. 0.10 = 10%)
DESCRIBE order_items;
-- 5 (7 rows: Lagos, Abuja, Port Harcourt, Ibadan, Kano, Enugu, Online)
SELECT DISTINCT city FROM stores;
-- 6 (5 methods: Card, Transfer, Cash, POS, USSD)
SELECT DISTINCT payment_method FROM orders;
-- 7 (Completed, Cancelled, Pending)
SELECT DISTINCT status FROM orders;
-- 8
SELECT product_name, unit_price,
       ROUND(unit_price * 1.075, 0) AS price_with_vat
FROM products;
-- 9
SELECT first_name, last_name, monthly_salary,
       monthly_salary * 12 AS annual_salary
FROM employees;
-- 10 (cheapest: Kora Tomato Paste and Tidal Tomato Paste at 600)
SELECT product_name, unit_price
FROM products
ORDER BY unit_price ASC
LIMIT 10;
-- 11
SELECT first_name, last_name, monthly_salary
FROM employees
ORDER BY monthly_salary DESC, last_name ASC;
-- 12 (latest order date is 2025-12-31)
SELECT *
FROM orders
ORDER BY order_date DESC
LIMIT 10;
```

```
-- 13 (most expensive per signup: Independence Day Sale 2025, about 8,065 per signup)
SELECT campaign_name, budget_ngn, new_signups,
       ROUND(budget_ngn / new_signups, 0) AS cost_per_signup
FROM marketing_campaigns
ORDER BY cost_per_signup DESC;
```

Data detective: **gender** has 6 different values: **F**, **M**, **Female**, **male**, **F** (with a hidden space in front), and **NULL** (missing). They should only be **M** and **F**. **city** has names with a **space in front** (**Lagos**) and some in **CAPITALS** (**LAGOS**). MySQL treats capitals and small letters as the same when comparing, so **DISTINCT** hides the capitals, but they are still stored that way. You'll clean all of this in Chapter 8.

Quiz answers

1. A **database** is a collection of tables. A **table** is data in rows and columns. A **row** is one record (one customer). A **column** is one type of information (city).
2. **SQL** is the language. **MySQL** is the database software that stores data and runs the SQL. **Workbench** is the program you type SQL into.
3. ***** means "all columns".
4. **AS** gives a column a new name (an **alias**) in the result.
5. **ASC** sorts smallest to largest (A to Z). **DESC** sorts largest to smallest (Z to A).
6. **Import** means file → database. **Export** means database → file.
7. The **;** tells MySQL where one statement ends, so you can write many statements in one tab.

Chapter 2: Filtering data with WHERE

Course days 3–4 · Finding exactly the rows you need

In Chapter 1, every query returned **all** the rows of a table. Real questions are almost always about **some** rows: "customers in Abuja", "orders in December", "products under #5,000". Picking rows by a condition is called **filtering**, and in SQL you filter with **WHERE**.

By the end of this chapter you can:

- filter with **=**, **<>**, **>**, **<**, **>=**, **<=**
- combine conditions with **AND**, **OR**, **NOT** and brackets
- use the shortcuts **IN**, **BETWEEN** and **LIKE**
- find missing values with **IS NULL**
- filter dates correctly

Start every session with:

```
USE brightmart;
```

2.1 The WHERE clause

```
SELECT *  
FROM customers  
WHERE city = 'Abuja';
```

Word by word:

- **WHERE** means "only keep the rows where..."
- **city = 'Abuja'** is the **condition**: the value in the **city** column must equal **Abuja**.

In plain English: "Show me every customer whose city is Abuja."

MySQL checks the condition **one row at a time**. If it's true for that row, the row is kept; if it's false, the row is thrown away.

Where WHERE goes: always **after FROM** and **before ORDER BY**:

```
SELECT    columns  
FROM      table  
WHERE     condition  
ORDER BY  column  
LIMIT    number;
```

Quotes: when to use them

- **Text and dates** go in **single quotes**: 'Abuja', '2025-12-24'.
- **Numbers** have **no quotes**: **unit_price > 50000**.
- Column and table names have no quotes.

A common beginner mistake is **WHERE city = Abuja**, without quotes. MySQL then thinks **Abuja** is a **column name** and shows **Unknown column 'Abuja'**.

Does capitalisation matter?

For text **values**, MySQL usually ignores upper and lower case: 'abuja', 'Abuja' and 'ABUJA' all match. But **spaces matter**: ' Abuja' (with a space in front) does **not** match 'Abuja'. Remember this for Chapter 8.

2.2 Comparison operators

Operator	Meaning	Example
=	equal to	status = 'Cancelled'
<> or !=	not equal to	status <> 'Completed'
>	greater than	unit_price > 100000
<	less than	unit_price < 2000
>=	greater than or equal to	quantity >= 3
<=	less than or equal to	discount <= 0.05

```
-- Products that sell for more than ₦100,000
SELECT product_name, category, unit_price
FROM products
WHERE unit_price > 100000
ORDER BY unit_price DESC;
-- Orders that were NOT completed
SELECT order_id, order_date, status
FROM orders
WHERE status <> 'Completed';
```

Conditions with calculations

You can compare a calculation, not just a column:

```
-- Products where profit per unit is more than ₦50,000
SELECT product_name, unit_price, unit_cost,
       unit_price - unit_cost AS profit_per_unit
FROM products
WHERE unit_price - unit_cost > 50000;
```

Why not WHERE profit_per_unit > 50000? MySQL runs **WHERE** **before** it creates the aliases in **SELECT**, so in **WHERE** the name **profit_per_unit** doesn't exist yet. Repeat the calculation instead.

2.3 Combining conditions: AND, OR, NOT

AND: both conditions must be true

```
-- Electronics that cost less than ₦30,000
SELECT product_name, unit_price
FROM products
WHERE category = 'Electronics'
      AND unit_price < 30000;
```

OR: at least one condition must be true

```
-- Customers in Kano or Kaduna
SELECT first_name, last_name, city
FROM customers
WHERE city = 'Kano'
      OR city = 'Kaduna';
```

NOT: reverses a condition

```
SELECT product_name, category
FROM products
WHERE NOT category = 'Groceries';
```

Brackets: the most important rule in this chapter

AND is always worked out **before** **OR**, just as $\times$ comes before $+$ in maths. That can give you the wrong answer without any error message.

Manager: "Show me Fashion or Groceries products that cost more than ₦5,000."

Wrong:

```
SELECT product_name, category, unit_price
FROM products
WHERE category = 'Fashion' OR category = 'Groceries' AND unit_price > 5000;
```

MySQL reads this as *Fashion (any price) OR (Groceries AND price > 5000)*, so cheap Fashion items sneak in.

Right:

```
SELECT product_name, category, unit_price
FROM products
WHERE (category = 'Fashion' OR category = 'Groceries')
      AND unit_price > 5000;
```

Rule: whenever you mix **AND** and **OR**, put brackets around the **OR** part.

2.4 Shortcuts: IN, BETWEEN, LIKE

IN: matches any value in a list

Instead of a long chain of ORs:

```
SELECT first_name, last_name, city
FROM customers
WHERE city IN ('Kano', 'Kaduna', 'Abuja');
```

NOT IN means "anything except these":

```
SELECT order_id, payment_method
FROM orders
WHERE payment_method NOT IN ('Cash', 'POS');
```

BETWEEN: a range, including both ends

```
SELECT product_name, unit_price
```

```
FROM products
WHERE unit_price BETWEEN 10000 AND 20000;
```

This is exactly the same as `unit_price >= 10000 AND unit_price <= 20000`. **Both ends are included.**

LIKE: pattern matching on text

Symbol	Means	Example	Matches
<code>%</code>	any number of characters (even none)	<code>'%Rice%'</code>	"Nova Rice 5kg", "Rice"
<code>_</code>	exactly one character	<code>'_ova%'</code>	"Nova Smartphone"

```
-- Products with "Baby" anywhere in the name
SELECT product_name FROM products WHERE product_name LIKE '%Baby%';
-- Products whose name STARTS with "Zenith"
SELECT product_name FROM products WHERE product_name LIKE 'Zenith%';
-- Products whose name ENDS with "Pack"
SELECT product_name FROM products WHERE product_name LIKE '%Pack';
-- Customers whose first name starts with "Ch"
SELECT first_name, last_name FROM customers WHERE first_name LIKE 'Ch%';
```

2.5 Missing values: NULL

NULL means **"no value"** or **"unknown"**. It isn't zero and it isn't an empty word; it's *nothing*. In BrightMart, 61 customers have no email address.

```
-- Customers with a missing email
SELECT customer_id, first_name, last_name, email
FROM customers
WHERE email IS NULL;
-- Customers who DO have an email
SELECT customer_id, first_name, email
FROM customers
WHERE email IS NOT NULL;
```

Never write `= NULL`. `WHERE email = NULL` returns **zero rows**, with no error, because nothing is ever "equal" to unknown. Always use `IS NULL` or `IS NOT NULL`.

2.6 Filtering dates

MySQL stores dates as `YYYY-MM-DD` (year-month-day). Write dates the same way, in quotes.

```
-- Orders on Christmas Eve 2024
SELECT * FROM orders WHERE order_date = '2024-12-24';
-- All orders in December 2024
SELECT * FROM orders
WHERE order_date BETWEEN '2024-12-01' AND '2024-12-31';
-- Orders from 2025 onwards
SELECT * FROM orders WHERE order_date >= '2025-01-01';
```

Two date functions you'll use all the time:

```
-- YEAR() pulls out the year; MONTH() pulls out the month number (1-12)
SELECT order_id, order_date
FROM orders
WHERE YEAR(order_date) = 2024
      AND MONTH(order_date) = 12;
```

2.7 The order MySQL actually runs your query

You **write** a query in this order: **SELECT** → **FROM** → **WHERE** → **ORDER BY** → **LIMIT**. MySQL **runs** it in this order:

1. **FROM**: find the table
2. **WHERE**: throw away rows that fail the condition
3. **SELECT**: pick and calculate columns (aliases are created here)
4. **ORDER BY**: sort (aliases can be used here)
5. **LIMIT**: cut to N rows

This explains why aliases work in **ORDER BY** but **not** in **WHERE**.

Exercises: Chapter 2

1. All products in the **Electronics** category.
2. Products with a unit price above ₦100,000, most expensive first.
3. Products costing between ₦5,000 and ₦20,000 (inclusive).
4. How many customers live in **Abuja**? (List them; the Output panel shows the number of rows returned.)
5. Customers whose email is missing.
6. All **Cancelled** orders.
7. All orders placed in **December 2024**.
8. Orders paid by **Card** or **Transfer** at **store 8** (Online). Watch your brackets.
9. Products whose name contains "**Baby**".
10. Employees whose role is **not** "Sales Associate".
11. Customers born **before 1970** (`date_of_birth`).
12. Customers who signed up through **Instagram** or **Google Ads** in **2025**.
13. Order lines (`order_items`) with a discount of **15%** (0.15) **and** a quantity of 4 or more.
14. Products in **Fashion** whose name ends with "**Shirt**" or "**Sneakers**".
15. Products that make **more than 35% profit margin**. Hint: `(unit_price - unit_cost) / unit_price > 0.35`.

Business challenge

Manager: "I want to run a premium campaign. Get me our high-value **Electronics and Home & Kitchen** products (price **at least ₦50,000**) with a margin **above 30%**, most profitable per unit first."

Write one query. Then write **two sentences** in plain English about what you found.

Quiz

1. Why is `WHERE email = NULL` wrong?
 2. Is `BETWEEN 10 AND 20` inclusive or exclusive?
 3. What's the difference between `'%Pack'` and `'Pack%'`?
 4. Why can't you use a `SELECT` alias inside `WHERE`?
-

Answers: Chapter 2

```
-- 1
SELECT * FROM products WHERE category = 'Electronics';
-- 2
SELECT product_name, unit_price FROM products
WHERE unit_price > 100000 ORDER BY unit_price DESC;
-- 3
SELECT product_name, unit_price FROM products
WHERE unit_price BETWEEN 5000 AND 20000;
-- 4 (283 rows)
SELECT * FROM customers WHERE city = 'Abuja';
-- 5 (61 rows)
SELECT * FROM customers WHERE email IS NULL;
-- 6 (790 rows)
SELECT * FROM orders WHERE status = 'Cancelled';
-- 7
SELECT * FROM orders WHERE order_date BETWEEN '2024-12-01' AND '2024-12-31';
-- 8
SELECT * FROM orders
WHERE (payment_method = 'Card' OR payment_method = 'Transfer')
    AND store_id = 8;
-- 9
SELECT product_name FROM products WHERE product_name LIKE '%Baby%';
-- 10
SELECT * FROM employees WHERE role <> 'Sales Associate';
-- 11
SELECT * FROM customers WHERE date_of_birth < '1970-01-01';
-- 12
SELECT * FROM customers
WHERE acquisition_channel IN ('Instagram', 'Google Ads')
    AND signup_date >= '2025-01-01';
-- 13
SELECT * FROM order_items WHERE discount = 0.15 AND quantity >= 4;
-- 14
SELECT product_name FROM products
WHERE category = 'Fashion'
    AND (product_name LIKE '%Shirt' OR product_name LIKE '%Sneakers');
-- 15
SELECT product_name, unit_price, unit_cost FROM products
WHERE (unit_price - unit_cost) / unit_price > 0.35;
-- Business challenge
SELECT product_name, category, unit_price, unit_cost,
    unit_price - unit_cost AS profit_per_unit,
    ROUND((unit_price - unit_cost) / unit_price * 100, 1) AS margin_pct
FROM products
WHERE category IN ('Electronics', 'Home & Kitchen')
    AND unit_price >= 50000
    AND (unit_price - unit_cost) / unit_price > 0.30
```

```
ORDER BY profit_per_unit DESC;
```

Sample business answer: "8 products qualify: 6 Electronics and 2 Gas Cookers. Laptops and smartphones lead; the Tidal Laptop earns about ₦219,000 profit per unit at a 43% margin, so the premium campaign should focus on laptops and smartphones."

Quiz: (1) NULL means unknown, so nothing is "equal" to it; use **IS NULL**. (2) Inclusive: both 10 and 20 are included. (3) **'%Pack'** means *ends with Pack*, and **'Pack%'** means *starts with Pack*. (4) **WHERE** runs before **SELECT**, so the alias doesn't exist yet.

Enjoyed the sample?

The full book continues for 27 more chapters

You've just finished the first two chapters of **Zero to Data Analyst**. The full book takes you all the way to job-ready:

Part	What you'll learn
A. SQL	GROUP BY, JOINS, functions, CTEs, window functions, cleaning data, and a business review project
B. Excel	Formulas, Power Query, PivotTables and an interactive dashboard
C. Power BI	Data modelling, DAX measures and an executive report
D. Python	pandas, cleaning, charts, automation and customer analysis (RFM and cohorts)
E. Career	Statistics, A/B testing, Tableau, GitHub, a capstone project, CV and interview preparation

Every chapter has exercises **with full answers**, and you practise on the same BrightMart dataset throughout.

Get the full book: <https://selar.com/651z512f16> (Selar, pay in naira), or message us on WhatsApp.